

OS 프로젝트 03 보고서

2023089507 김현중

목차

1. COW 구현
 - 1-1. COW 분석
 - 1-2. 코드 분석
 - 1-3. 구현
 - 1-4. 결과 분석
2. Doubly Indirect Block 구현
 - 2-1. Doubly Indirect Block 분석 2-2. 코드 분석
 - 2-3. 구현
 - 2-4. 결과 분석
3. Symbolic Link 구현
 - 3-1. Symbolic Link 분석
 - 3-2. 코드 분석
 - 3-3. 구현
 - 3-4. 결과 분석

COW 구현

COW 분석

Copy-On-Write(COW)는 페이지를 공유하는 방식으로 메모리 사용을 최적화하는 기법이다. 페이지가 공유되다가 쓰기 작업이 발생하면, 해당 페이지를 복사하여 새로운 페이지를 할당하고, 쓰기 작업은 새 페이지에 적용된다. 이를 통해 불필요한 메모리 복사를 방지할 수 있다. 프로세스 포크가 발생하면 부모 프로세스의 페이지 테이블을 복사하여 자식 테이블의 페이지 테이블로 사용한다. 이 때 PTE_W 비트를 끄고 PTE_COW 비트를 켜서 COW로 페이지가 공유돼 있음을 OS에 알려준다. 이후 해당 페이지를 수정하려고 하면 PTE_W 비트가 켜져있기 때문에 페이지 폴트가 발생하고 핸들러를 통해 페이지를 할당하고 값을 복사하여 해당 페이지에 쓰기 작업을 수행한다. 이 과정에서는 PTE_W 비트를 켜준다. COW는 유저 페이지에 대해서만 적용되기 때문에 실제 구현에서는 유저 메모리를 다루는 함수에 대해서만 수정이 필요하다.

uvmcopy 분석

```
int
uvmcopy(pagetable_t old, pagetable_t new, uint64 sz)
{
    for(i = 0; i < sz; i += PGSIZE){
        ...
        pa = PTE2PA(*pte);
        flags = PTE_FLAGS(*pte);
        if((mem = kalloc()) == 0)
            goto err;
        memmove(mem, (char*)pa, PGSIZE);
        if(mappages(new, i, PGSIZE, (uint64)mem, flags) != 0){
            kfree(mem);
            goto err;
        }
    }
}
```

```

    return 0;

err:
    uvmunmap(new, 0, i / PGSIZE, 1);
    return -1;
}

```

기존의 uvmcopy는 모든 page에 대해 새롭게 물리주소를 할당(kalloc)하고 memmove로 값을 복사했다. COW에서는 이를 물리주소를 공유하게 변경해야 한다. uvmcopy는 유저 페이지에 대해서만 동작하기 때문에 따로 유저 페이지 flag를 확인할 필요는 없다.

uvmunmap 분석

```

void
uvmunmap(pagetable_t pagetable, uint64 va, uint64 npages, int do_free)
{
    ...
    for(a = va; a < va + npages*PGSIZE; a += PGSIZE){
        ...
        if(do_free){
            uint64 pa = PTE2PA(*pte);
            kfree((void *)pa);
        }
        *pte = 0;
    }
}

```

uvmunmap은 가상주소에 매핑된 페이지를 언맵하는 함수이다. do_free가 1이면 해당 페이지를 해제한다. COW에서는 reference counting을 사용해야 하기 때문에 do_free 1일 때 kfree를 바로 호출하는 것이 아닌 reference count가 0이 될 때만 kfree를 호출해야 한다.

copyout 분석

```

int
copyout(pagetable_t pagetable, uint64 dstva, char *src, uint64 len)
{
    uint64 n, va0, pa0;
    pte_t *pte;

    while(len > 0){
        ...
        pte = walk(pagetable, va0, 0);
        if(pte == 0 || (*pte & PTE_V) == 0 || (*pte & PTE_U) == 0 ||
           (*pte & PTE_W) == 0)
            return -1;
        pa0 = PTE2PA(*pte);
        n = PGSIZE - (dstva - va0);
        if(n > len)
            n = len;
    }
}

```

```

    memmove((void *)(pa0 + (dstva - va0)), src, n);
    ...
}
return 0;
}

```

copyout은 유저 페이지에 값을 쓰는 함수이다. COW에서는 페이지 테이블이 페이지를 공유할 수 있기에 페이지를 공유하고 있는지 체크하고 페이지를 공유하고 있으면 페이지를 복사하여 새롭게 할당한 후 값을 쓰도록 해야 한다.

uvmmalloc 및 uvmfirst 분석

```

void
uvmfirst(pagetable_t pagetable, uchar *src, uint sz)
{
    char *mem;

    if(sz >= PGSIZE)
        panic("uvmfirst: more than a page");
    mem = kalloc();
    increment_ref((uint64)mem);
    memset(mem, 0, PGSIZE);
    mappages(pagetable, 0, PGSIZE, (uint64)mem, PTE_W|PTE_R|PTE_X|PTE_U);
    memmove(mem, src, sz);
}

uint64
uvmmalloc(pagetable_t pagetable, uint64 oldsz, uint64 newsz, int xperm)
{
    ...
    for(a = oldsz; a < newsz; a += PGSIZE){
        mem = kalloc();
        if(mem == 0){
            uvmdealloc(pagetable, a, oldsz);
            return 0;
        }
        memset(mem, 0, PGSIZE);
        if(mappages(pagetable, a, PGSIZE, (uint64)mem, PTE_R|PTE_U|xperm) != 0){
            kfree(mem);
            uvmdealloc(pagetable, a, oldsz);
            return 0;
        }
    }
    return newsz;
}

```

COW에서는 reference counting을 지원해야하기에 유저 페이지를 할당하는 함수에서 reference count를 증가시켜야 한다. uvmfirst와 uvmmalloc 함수가 유저 메모리를 할당하는 함수이기에 여기서 reference count를 증가하게 한다.

usertrap 분석

page fault가 발생했을 때 처리하는 usertrap 함수에서 COW를 처리해야 한다. usertrap에서 COW를 처리해야 하는지 PTE_COW 비트와 PTE_W 비트를 확인하고 적절히 처리하는 코드를 추가해야 한다.

COW 구현

reference counting

총 PAGE의 개수는 전체 주소 / PAGE_SIZE로 구할 수 있다. 페이지 개수만큼의 길이를 갖는 배열을 만들어 각 페이지의 reference count를 저장한다.

```
#define N_PHY_PAGES ((PHYSTOP) >> PGSHIFT)
static int pgref[N_PHY_PAGES];
```

그리고 reference count를 증가시키는 함수와 감소시키는 함수, 그리고 reference count를 반환하는 함수를 구현한다. 그리고 reference count에 접근할 때는 pgref_lock을 사용해 동기화한다. decrement_ref 함수에서 reference count가 0이 되면 해당 페이지를 해제한다.

```
void increment_ref(uint64 pa){
    if (pa >= PHYSTOP) {
        panic("increment_ref: invalid pa");
    }
    acquire(&pgref_lock);
    ++pgref[pa >> PGSHIFT];
    //printf("inc: pgref[%lu]=%d\n", pa, r);
    release(&pgref_lock);
}

void decrement_ref(uint64 pa){
    if (pa >= PHYSTOP) {
        panic("decrement_ref: invalid pa");
    }
    acquire(&pgref_lock);
    int r = --pgref[pa >> PGSHIFT];
    //printf("dec: pgref[%lu]=%d\n", pa, r);
    release(&pgref_lock);
    if(r == 0) kfree((void *)pa);
}

int get_ref(uint64 pa){
    if (pa >= PHYSTOP) {
        panic("decrement_ref: invalid pa");
    }
    acquire(&pgref_lock);
    int r = pgref[pa >> PGSHIFT];
    release(&pgref_lock);
    return r;
}
```

uvmcopy 수정

```

int
uvmcopy(pagetable_t old, pagetable_t new, uint64 sz)
{
    ...
    for(i = 0; i < sz; i += PGSIZE){
        ...
        *pte &= ~PTE_W;
        *pte |= PTE_COW;
        pa = PTE2PA(*pte);
        flags = PTE_FLAGS(*pte);
        if(mappages(new, i, PGSIZE, pa, flags) != 0){
            goto err;
        }
        increment_ref(pa);
    }
    sfence_vma();
    return 0;
    ...
}

```

메모리를 할당하는 코드를 삭제하고 페이지를 공유하게 바꿨다. PTE_W 비트를 끄고 PTE_COW 비트를 켜서 COW로 페이지가 공유됨을 알린다. 그리고 페이지를 공유하기 때문에 물리주소를 할당할 필요가 없고, 기존의 물리주소를 그대로 사용한다. 그리고 해당 페이지의 reference count를 증가시킨다. sfence_vma();는 pte가 변경 되었으므로 TLB를 갱신하기 위해 호출한다.

uvmunmap 수정

```

void
uvmunmap(pagetable_t pagetable, uint64 va, uint64 npages, int do_free)
{
    ...
    for(a = va; a < va + npages*PGSIZE; a += PGSIZE){
        ...
        if(do_free){
            uint64 pa = PTE2PA(*pte);
            decrement_ref(pa);
        }
        *pte = 0;
    }
}

```

uvmunmap에서 do_free가 1일 때 kfree를 호출하는 것이 아닌 decrement_ref를 호출하도록 수정했다. decrement_ref는 reference count가 0이 될 때만 kfree를 호출하기 때문에 페이지가 공유되고 있어도 안전하게 동작한다.

copyout 수정

```

int
copyout(pagetable_t pagetable, uint64 dstva, char *src, uint64 len)
{
    ...
    while(len > 0){
        ...
        if(pte == 0 || (*pte & PTE_V) == 0 || (*pte & PTE_U) == 0 ||
            ((*pte & PTE_W) == 0 && (*pte & PTE_COW) == 0))
            return -1;
        pa0 = PTE2PA(*pte);
        n = PGSIZE - (dstva - va0);
        if(n > len)
            n = len;

        pa0 = re_page(pagetable, pte, va0);
        ...
    }
    return 0;
}

```

pte가 올바른지 확인하는 부분에서 PTE_W 비트가 꺼져있고 PTE_COW 비트가 켜져있는 경우를 추가했다. 그리고 re_page 함수를 호출하여 물리 주소를 새롭게 얻는데 re_page는 페이지가 공유되고 있으면 페이지를 복사하여 새롭게 할당하고, 그렇지 않으면 기존의 물리 주소를 그대로 사용한다.

uvmfirst 및 uvmalloc 수정

```

void
uvmfirst(pagetable_t pagetable, uchar *src, uint sz)
{
    ...
    mem = kalloc();
    increment_ref((uint64)mem);
    memset(mem, 0, PGSIZE);
    mappages(pagetable, 0, PGSIZE, (uint64)mem, PTE_W|PTE_R|PTE_X|PTE_U);
    memmove(mem, src, sz);
}

uint64
uvmalloc(pagetable_t pagetable, uint64 oldsz, uint64 newsz, int xperm)
{
    ...
    for(a = oldsz; a < newsz; a += PGSIZE){
        ...
        increment_ref((uint64)mem);
    }
    return newsz;
}

```

uvmfirst와 uvmalloc에서 물리 주소를 할당할 때 kalloc을 호출한 후 increment_ref를 호출하여 reference count를 증가시킨다.

usertrap 수정

```
void
usertrap(void)
{
    ...
    if(r_scause() == 8){
        ...
    } else if(r_scause() == 15){
        uint64 fault_va = r_stval();
        if(pagefault_handler(p->pagetable, fault_va) < 0){
            printf("page fault: unexpected scause 0x%lx pid=%d\n", r_scause(), p->pid);
            printf("          sepc=0x%lx stval=0x%lx\n", r_sepc(), r_stval());
            setkilled(p);
        }
    }
    else if((which_dev = devintr()) != 0){
        // ok
    } else {
        ...
    }
    ...
}
```

r_scause() == 15인 경우 페이지 폴트가 발생한 경우이다. 이 때는 pagefault_handler 함수를 호출하여 페이지 폴트를 처리한다. pagefault_handler는 PTE_COW 비트와 PTE_W 비트를 확인하여 COW 페이지 폴트인지 확인하고, COW 페이지 폴트인 새롭게 페이지를 할당한다. 그 외의 경우는 에러 메시지를 출력하고 프로세스를 종료한다.

page_change 함수 구현

```
uint64 page_change(pagetable_t pagetable, pte_t *pte, uint64 va0){
    uint64 oldpa = PTE2PA(*pte);
    uint64 pa = oldpa;
    if(get_ref(oldpa) > 1){
        pa = (uint64)kalloc();
        if(pa == 0) return 0;
        memmove((void*)pa, (void*)oldpa, PGSIZE);
        decrement_ref(oldpa);
        increment_ref(pa);
    }
    uint64 flags = PTE_FLAGS(*pte);
    flags |= PTE_W;
    flags &= ~PTE_COW;
    *pte = PA2PTE(pa) | flags;
    sfence_vma();
}
```

```
    return pa;
}
```

page_change 함수는 페이지가 공유되고 있는 경우 새롭게 페이지를 할당하고 값을 복사하여 해당 페이지에 쓰기 작업을 수행한다. 만약 페이지가 공유되고 있지 않다면 기존의 물리 주소를 그대로 사용한다. 페이지가 공유되고 있는지는 reference count로 확인할 수 있다. 그리고 PTE_W 비트를 켜고 PTE_COW 비트를 끈다. pte가 수정됐으니 sfence_vma를 호출하여 TLB를 갱신한다.

re_page 함수 구현

```
uint64 re_page(pa_t pa, pte_t *pte, uint64 va){
    return (*pte & PTE_W)?
        PTE2PA(*pte) :
        page_change(pa, pte, va);
}
```

re_page는 PTE_W 비트가 켜져있으면 기존의 물리 주소를 그대로 사용하고, PTE_W 비트가 꺼져있고 PTE_COW 비트가 켜져있으면 page_change 함수를 호출하여 페이지를 새롭게 할당하고 해당 물리주소를 반환한다.

****pagefault_handler** 구현**

```
```C
int
pagefault_handler(pa_t pa, uint64 va){
 pte_t *pte;
 va = PGROUNDDOWN(va);
 if ((pte = walk(pa, va, 0)) == 0 || (*pte & PTE_V) == 0) return -1;
 else if((*pte & PTE_COW) == 0) return -1;
 if(page_change(pa, pte, va) == 0) return -1;
 return 0;
}
```

pagefault\_handler 함수는 페이지 폴트가 발생했을 때 호출된다. 이 함수는 페이지 테이블에서 해당 가상 주소의 pte를 찾고, PTE\_COW 비트가 켜져있는지 확인한다. PTE\_COW 비트가 켜져있으면 page\_change 함수를 호출하여 페이지를 새롭게 할당하고, 그렇지 않으면 -1을 반환한다.

### 결과 분석

```
init: starting sh
$ cowtest
simple: ok
simple: ok
three: ok
three: ok
three: ok
file: ok
ALL COW TESTS PASSED
```



cowtest를 실행하여 COW 구현이 올바르게 동작하는지 확인했다. simple, three, file 테스트 모두 성공했다. simpletest는 물리 메모리의 2/3을 할당하고 fork가 정상적으로 동작하는지 확인한다. threetest는 3개의 프로세스를 fork하고 각 프로세스가 서로 다른 페이지를 수정하는지 확인하는 테스트이다. fork를 2번 실행하고 3번째 프로세스는 배열을 9999로 초기화하고 두 번째 프로세스는 배열을 pid로 첫번째도 자신의 pid로 초기화한다. 이 때 첫번째 프로세스에서 배열이 pid로 초기화하는지 확인한다. Filetest는 read함수로 파일을 읽음으로서 filediscriptor number를 가져오는데 이를 통해 copyout이 올바르게 동작하는지 확인한다. 올바르게 동작한다면 fork된 프로세스마다 서로 다른 파일 디스크립터를 가져오게 된다. 모든 테스트가 성공적으로 통과했다. COW 구현이 올바르게 동작한다는 것을 확인할 수 있다.

## Doubly Indirect Block 구현

### Doubly Indirect Block 분석

파일 시스템에서 Block Addressing의 구조는 direct 12개 indirect 1개로 이뤄져있다. direct 주소는 data로 직접 연결되는 주소를 갖고 있고 indirect 주소는 address 256개를 가르키는 블록을 가르키고 있다. 여기서 추가해야 하는 것은 Doubly Indirect 구조로 Indirect 블록 256개를 갖고 있는 블록을 가르키는 주소를 추가해야 한다.

## 코드 분석

### dinode 및 inode 분석

```
struct dinode {
 ...
 uint addrs[NDIRECT+1]; // Data block addresses (NDIRECT direct, 1 indirect)

 struct inode {
 ...
 uint addrs[NDIRECT+1];
 };
};
```

해당 구조체에서 핵심부분은 addrs이다. addrs는 direct 주소 12개와 indirect 주소 1개를 갖고 있다. doubly indirect block을 추가하기 위해서는 addrs를 1개 더 늘려서 총 2개의 indirect 주소를 갖도록 해야 한다.

## bmap 분석

```
static uint
bmap(struct inode *ip, uint bn)
{
 uint addr, *a;
 struct buf *bp;

 if(bn < NDIRECT){
 if((addr = ip->addrs[bn]) == 0){
 addr = balloc(ip->dev);
 if(addr == 0)
 return 0;
 ip->addrs[bn] = addr;
 }
 return addr;
 }
```

```

}
bn -= NDIRECT;

if(bn < NINDIRECT){
 // Load indirect block, allocating if necessary.
 if((addr = ip->addrs[NDIRECT]) == 0){
 addr = balloc(ip->dev);
 if(addr == 0)
 return 0;
 ip->addrs[NDIRECT] = addr;
 }
 bp = bread(ip->dev, addr);
 a = (uint*)bp->data;
 if((addr = a[bn]) == 0){
 addr = balloc(ip->dev);
 if(addr){
 a[bn] = addr;
 log_write(bp);
 }
 }
 brelse(bp);
 return addr;
}
panic("bmap: out of range");
}

```

bmap은 block number를 실제 disk 주소에 매핑하는 함수이다. Direct에 대해 매핑돼 있고 주소가 할당돼있지 않으면 balloc함수를 사용해 할당한다. Indirect로 매핑될 때는 data 주소들을 저장하기 위한 block을 할당하고 bread를 통해 해당 block을 읽어온다 block에서 bn에 해당하는 주소가 할당돼 있지 않으면 balloc을 통해 할당하고 log\_write를 통해 해당 block을 기록한다. 그리고 bread로 읽어온 block은 brelse를 통해 해제한다. doubly indirect block을 추가하려면 Indirect로 매핑되는 부분의 코드를 사용해 코드를 추가해야 한다. Indirect block을 할당하는 과정은 balloc을 통해 할당하고 해당 블록에 데이터 주소를 저장하는 과정이었다. Doubly indirect block은 balloc을 통해 indiret block을 두번 할당한다.

### itrunc 분석

```

void
itrunc(struct inode *ip)
{
 int i, j, k;
 struct buf *bp, *bp2;
 uint *a, *b;

 for(i = 0; i < NDIRECT; i++){
 if(ip->addrs[i]){
 bfree(ip->dev, ip->addrs[i]);
 ip->addrs[i] = 0;
 }
 }

 if(ip->addrs[NDIRECT]){

```

```

 bp = bread(ip->dev, ip->addrs[NDIRECT]);
 a = (uint*)bp->data;
 for(j = 0; j < NINDIRECT; j++){
 if(a[j])
 bfree(ip->dev, a[j]);
 }
 brelse(bp);
 bfree(ip->dev, ip->addrs[NDIRECT]);
 ip->addrs[NDIRECT] = 0;
}
ip->size = 0;
iupdate(ip);
}

```

itrunc는 inode의 데이터를 모두 해제하는 함수이다. Direct 주소에 대해 bfree를 호출하여 해당 주소를 해제하고, Indirect 주소에 대해서도 bread를 통해 해당 block을 읽어온 후, 해당 block의 주소들을 bfree를 통해 해제한다. 그리고 Indirect 주소를 해제하고 inode의 크기를 0으로 설정한 후 iupdate를 호출하여 inode를 업데이트한다. doubly indirect block을 추가하려면 indirect block 두 번거쳐서 해제해야 하기 때문에 Indirect 주소를 해제하는 부분을 두 번 추가해야 한다. 이는 이중 for문을 사용할 것이다.

## doubly indirect block 구현

### inode 및 dinode 수정

```

struct inode {
 ...
 uint addrs[NDIRECT+2]; // Data block addresses
};
struct dinode {
 ...
 uint addrs[NDIRECT+2]; // Data block addresses
};

```

주소를 하나 추가하여 doubly indirect block을 추가한다.

### bmap 수정

```

static uint
bmap(struct inode *ip, uint bn)
{
 bn -= NINDIRECT;
 if(bn < NDINDIRECT){
 if((addr = ip->addrs[NIRECT]) == 0){
 addr = balloc(ip->dev);
 if(addr == 0)
 return 0;
 ip->addrs[NIRECT] = addr;
 }
 bp = bread(ip->dev, addr);
 }
}

```

```

 a = (uint*)bp->data;
 uint i1 = bn >> 8;
 uint i2 = bn & 0xff;
 if((addr = a[i1]) == 0){
 addr = balloc(ip->dev);
 if(addr == 0) return 0;
 a[i1] = addr;
 log_write(bp);
 }
 brelse(bp);
 bp = bread(ip->dev, addr);
 a = (uint*)bp->data;
 if((addr = a[i2]) == 0){
 addr = balloc(ip->dev);
 if(addr){
 a[i2] = addr;
 log_write(bp);
 }
 }
 brelse(bp);
 return addr;
}
panic("bmap: out of range");
}

```

bn이 Double indirect block에 할당되는 경우 첫번째 block과 두번째 block에 대한 인덱스가 필요하다. 첫번째 block은 256개의 주소를 갖고 있고, 두번째 block은 256개의 주소를 갖고 있다. 따라서 bn을 256으로 나눈 몫을 i1로, 나머지를 i2로 설정한다. 그리고 첫번째 block에 대한 주소를 할당하고, 해당 주소를 bread를 통해 읽어온 후, 두번째 block에 대한 주소를 할당한다. 두번째 블록의 a[i2]는 data에 대해서 할당이 이뤄진다.

### itrunc 수정

```

void
itrunc(struct inode *ip)
{
 ...
 if(ip->addrs[NIRECT]){
 bp = bread(ip->dev, ip->addrs[NIRECT]);
 a = (uint*)bp->data;
 for(j = 0; j < NINDIRECT; j++){
 if(a[j]){
 bp2 = bread(ip->dev, a[j]);
 b = (uint*)bp2->data;
 for(k = 0; k < NINDIRECT; k++){
 if(b[k]){
 bfree(ip->dev, b[k]);
 }
 }
 brelse(bp2);
 bfree(ip->dev, a[j]);
 }
 }
 }
}

```

doubly indirect block을 해제하는 코드를 추가했다. 이중 for문으로 각각 256까지 반복한다. 첫번째 for문에서는 주소를 담고 있는 block을 두 번째 for문에서는 데이터를 가르키는 block을 가져오고 해당 주소가 할당돼 있으면 bfree를 호출하여 해당 주소를 해제한다. 그리고 두 번째 for문이 끝나면 첫번째 for문에서 접근한 block을 해제하고, 첫 번째 for문이 끝나면 doubly indirect block을 해제한다.

[illegible]

13 / 17

**sys\_open** 분석

```
uint64
sys_open(void)
{
 char path[MAXPATH];
 int fd, omode;
 struct file *f;
 struct inode *ip;
 int n;

 argint(1, &omode);
 if((n = argstr(0, path, MAXPATH)) < 0)
 return -1;

 begin_op();

 if(omode & O_CREATE){
 ip = create(path, T_FILE, 0, 0);
 if(ip == 0){
 end_op();
 return -1;
 }
 }
 else {
 if((ip = namei(path)) == 0){
 end_op();
 return -1;
 }
 ilock(ip);
 if(ip->type == T_DIR && omode != O_RDONLY){
 iunlockput(ip);
 end_op();
 return -1;
 }
 }

 if(ip->type == T_DEVICE && (ip->major < 0 || ip->major >= NDEV)){
 iunlockput(ip);
 end_op();
 return -1;
 }

 if((f = filealloc()) == 0 || (fd = fdalloc(f)) < 0){
 if(f)
 fileclose(f);
 iunlockput(ip);
 end_op();
 return -1;
 }

 if(ip->type == T_DEVICE){
 f->type = FD_DEVICE;
```

```

 f->major = ip->major;
} else {
 f->type = FD_INODE;
 f->off = 0;
}
f->ip = ip;
f->readable = !(omode & O_WRONLY);
f->writable = (omode & O_WRONLY) || (omode & O_RDWR);

if((omode & O_TRUNC) && ip->type == T_FILE){
 itrunc(ip);
}

iunlock(ip);
end_op();

return fd;
}

```

sys\_open 함수는 파일을 열 때 호출되는 시스템 콜이다. 이 함수는 파일 경로와 모드를 인자로 받아서 해당 파일을 열고 파일 디스크립터를 반환한다. 만약 O\_CREATE 플래그가 설정되어 있으면 create 함수를 호출하여 새로운 파일을 생성하고, 그렇지 않으면 namei 함수를 호출하여 해당 경로의 inode를 찾는다. 만약 O\_NOFOLLOW 플래그가 설정되어 있지 않고, 파일이 심볼릭 링크인 경우, 심볼릭 링크의 내용을 읽어서 실제 파일 경로를 찾는다. 이 과정은 심볼릭 링크가 여러 번 중첩될 수 있기 때문에 최대 10번까지 반복한다. 만약 심볼릭 링크가 계속해서 발견되면 -1을 반환하고 종료한다. 이후 파일 디스크립터를 할당하고, 파일 타입에 따라 설정을 한다. 만약 O\_TRUNC 플래그가 설정되어 있고, 파일 타입이 T\_FILE인 경우 itrunc 함수를 호출하여 해당 파일의 내용을 비운다.

## sys\_open

```

uint64
sys_open(void)
{
 ...
 if((omode & O_NOFOLLOW) == 0 && ip->type == T_SYMLINK){
 int cnt = 10;
 while (ip->type == T_SYMLINK && --cnt){
 readi(ip, 0, (uint64)path, 0, ip->size);
 iunlockput(ip);
 ip = namei(path);
 if(ip == 0){
 end_op();
 return -1;
 }
 ilock(ip);
 }
 if(ip->type == T_SYMLINK) {
 iunlockput(ip);
 end_op();
 return -1;
 }
 }
}

```

```

 }
 ...
}

```

sys\_open 함수는 파일을 열 때 호출되는 시스템 콜이다. 이 함수는 파일 경로와 모드를 인자로 받아서 해당 파일을 열고 파일 디스크립터를 반환한다. O\_NOFOLLOW가 없을 때 심볼릭 링크에 적힌 링크를 따라가는 코드를 추가했다. O\_NOFOLLOW 플래그가 설정되어 있지 않고, 파일이 심볼릭 링크인 경우, 심볼릭 링크의 내용을 읽어서 실제 파일 경로를 찾는다. 이 과정은 심볼릭 링크가 여러 번 중첩될 수 있기 때문에 최대 10번까지 반복한다. 만약 심볼릭 링크가 계속해서 발견되거나 경로가 올바르지 않으면 -1을 반환하고 종료한다.

### symlink 시스템콜 구현

```

uint64
symlink(char *target, char *path){
 struct inode *ip;
 int n1 = strlen(target);
 begin_op();
 if((ip = create(path, T_SYMLINK, 0, 0)) == 0){
 end_op();
 return -1;
 }
 int r = writei(ip, 0, (uint64)target, 0, n1 + 1);
 if(r != n1 + 1) {
 ip->nlink = 0;
 iupdate(ip);
 iunlockput(ip);
 end_op();
 return -1;
 }
 iupdate(ip);
 iunlockput(ip);

 end_op();
 return 0;
}
uint64 sys_symlink(){
 char target[MAXPATH + 1], path[MAXPATH + 1];
 if(argstr(0, target, MAXPATH) < 0 || argstr(1, path, MAXPATH) < 0)
 return -1;
 return symlink(target, path);
}

```

symlink 시스템 콜은 심볼릭 링크를 생성하는 함수이다. 먼저 target과 path를 인자로 받아서 해당 경로에 심볼릭 링크를 생성한다. create 함수를 통해 심볼릭 링크를 생성하고, writei 함수를 통해 심볼릭 링크의 내용을 target으로 설정한다. create 함수에는 T\_SYMLINK를 인자로 넣어 타입을 T\_SYMLINK로 설정한다. **결과 분석**

```

Start: test symlinks
test symlinks: ok

```



```
Start: test concurrent symlinks
test concurrent symlinks: ok
```

symlinktest를 실행하여 심볼릭링크가 올바르게 동작하는지 확인했다. testsymlink는 심볼릭 링크를 생성하고 해당 링크를 통해 파일에 접근하는 테스트이다. 또한 체이닝 링크, 사이클이 발생한 링크, 잘못된 경로가 있는 링크 등을 잘 핸들링하는지 확인한다. concur는 여러 프로세스가 동시에 심볼릭 링크를 생성하고 접근하는 테스트이다. 동시성 문제가 발생하는지를 확인한다. 두 테스트 모두 성공적으로 통과했다. 심볼릭 링크를 성공적으로 구현했음을 알 수 있다.

## 결론

이번 프로젝트에서는 COW, Doubly Indirect Block, Symbolic Link를 구현했다. COW는 페이지를 공유하는 방식으로 메모리 사용을 최적화하는 기법으로, 페이지 폴트가 발생했을 때 페이지를 새롭게 할당하고 값을 복사하여 쓰기 작업을 수행하도록 했다. Doubly Indirect Block은 indirect 블록을 두 번 거쳐서 데이터를 저장하는 구조로, 파일 시스템에서 더 많은 데이터를 저장할 수 있도록 했다. Symbolic Link는 파일 시스템에서 다른 파일이나 디렉토리의 경로를 가리키는 특수한 파일로, symlink 시스템 콜을 통해 심볼릭 링크를 생성하고 해당 링크를 통해 파일에 접근할 수 있도록 했다. 모든 구현이 성공적으로 동작하며 테스트를 통과했다.